Readable Is Not Used, Used Is Not Needed: An Inference-Only Audit of Continuous-Thought Reasoners

Anonymous Author(s)
Affiliation
email

Abstract

Some language models reason in a handful of hidden continuous vectors ("thoughts") instead of written steps. People want to monitor those thoughts for safety, but nobody has checked, on released checkpoints, which thoughts the model actually uses, which it cannot do without, and whether reading a thought tells you anything the model acted on. We built a one-command audit that answers these questions without any training, and ran it on four released checkpoints (CODI on GPT-2, CODI on LLaMA-3.2-1B, SIM-CoT on LLaMA-3.2-1B, and a Coconut GPT-2 retrain), four datasets, and 28 preregistered Kaggle runs. For the CODI family we find: (i) disturbing a thought and using a thought are different things, and a thought carries a value into the answer only on problems with three or more steps; (ii) exactly one or two thoughts do the carrying, they hand the value to the next position's memory, and the last thought never matters; (iii) the larger model uses its carrier thought but barely needs it, while the model's memory of the question is the single most necessary site; (iv) a probe on any thought, including the unused last one, reads the question well, so a monitor can look accurate while reading nothing the model computed; (v) the thoughts predict wrong answers no better than the model's own confidence; and (vi) the carrier thought can be written to from outside: transplanting a natural thought that holds the right intermediate value repairs 15–23% of failed multi-step problems against a 3% control, with 5–7% collateral damage at 1B. Coconut is the opposite case: its thoughts carry values on one-step problems, every pair is necessary (45–49 points), its last position is in use, and it cannot be written to. All results reproduce byte for byte across sessions. The audit costs about one GPU-hour per model and tells you, before you trust a monitor, which kind of model you have.

1 Introduction

A model that reasons in text can be watched: the steps are tokens, and a monitor can read them. Several recent methods instead train models to reason in a short run of continuous vectors that nobody can read directly, among them Coconut (Hao et al., 2024), CODI (Shen et al., 2025) and SIM-CoT (Wei et al., 2026). The usual response is to read the vectors with a probe (Ramjee et al., 2026). That response rests on three assumptions: that the probed vector holds something the model computed, that the model goes on to use it, and that the model would answer differently without it. None of these has been tested on the released checkpoints with a method that another lab could rerun.

This paper tests all three with interventions only. We never train or fine-tune a model. We take each checkpoint as released, swap, remove and read its thoughts, and measure what the answer does.

37th Conference on Neural Information Processing Systems (NeurIPS 2023).

Everything was preregistered in a running ledger before each run, scored against the prediction afterwards, and every number in this paper comes from a committed artifact.

Contributions.

1. **A method that separates three questions a probe conflates.** "Twin" problems that differ in one number tell us whether a thought *carries* a value (transport). Masking tells us whether the model *needs* it (necessity). Probes tell us whether it is *readable*, and the twins tell us whether what a probe reads is *confirmed by behaviour* (faithfulness). We also separate the two routes by which a thought can act: the state it feeds forward and the keys and values it leaves in memory.
2. **A standardised card and a comparison tool.** One command produces one card per (model, dataset, GPU session) with fixed thresholds, five checks, a regime and a monitor-placement verdict, plus fingerprints that let two sessions be compared bit for bit. We ran it on four models, four datasets and two Kaggle accounts.
3. **Findings about the CODI family** (Sections 5.1–5.3): a chain-length gate on transport; a trained carry-and-store ladder with an inert last thought; sufficiency without necessity at 1B; the question register as the most necessary site; and readable-but-unfaithful thoughts under two probe families.
4. **Three uses of the map** (Sections 6.1–6.3): a prompt-leakage control that any latent-monitor evaluation should include; a negative result on error detection from thoughts; and a positive one, latent repair by transplanting a natural thought that holds the right value, with a value from the answer key and, more weakly, from the model's own explicit chain plus a calculator.
5. **A contrast architecture** (Section 7): on Coconut the audit returns the opposite profile on almost every axis, which shows that the CODI results are properties of its training, not of continuous thoughts in general.

We also report two corrections to our own earlier readings and one to the literature: the released CODI-1B checkpoint scores 55.5% on GSM8K, which matches its paper's main figure of 55.6%; the 51.9% quoted by later papers is an ablation row.

2 Related work

Continuous-thought models. Coconut (Hao et al., 2024) feeds the last hidden state back as the next input embedding and replaces reasoning steps with such thoughts by a curriculum. CODI (Shen et al., 2025) distils an explicit-chain teacher into six thoughts through a projector, in one stage. SIM-CoT (Wei et al., 2026) adds a step-level auxiliary decoder during training. We study released CODI and SIM-CoT checkpoints and a third-party Coconut retrain, since no official Coconut GSM8K checkpoint exists.

Causal analyses of latent thoughts. Cywiński et al. (2025) patched single latent vectors of a self-trained CODI-1B with averages from prompts holding different intermediate values, on twenty templates with two steps, and found the third and fifth vectors store the step values; they left longer problems open. SCIT (Ding et al., 2026) swapped value-cache layers on same-template pairs. Li et al. (2026) overwrite single steps with zeros and report flip rates on the official checkpoints. Aswal et al. (2026) show that decodable patterns appear in controls without recurrence and call for causal tests. Jin et al. (2026) find accuracy retained when latent computation is removed. Our twins extend Cywiński et al. (2025) to natural data, three checkpoints and chains of three or more steps, add the two-route decomposition, necessity and the monitor layer, and run everything from released weights.

Monitoring and probes. Ramjee et al. (2026) detect misaligned reasoning in a backdoored Coconut-style model by probing early states, with the trigger in the prompt. Ferreira et al. (2026) argue monitorability depends on access and task structure. That decodability is not causality is established elsewhere (Open Problems, 2025; Decodability, 2026; Single-position, 2026); we measure it per slot on latent reasoners.

Interventions and repair. Chang et al. (2026) improve accuracy with training-free geometric edits of cached thoughts, without aiming a value at a problem. Huang and Qi (2026) find synthetic steering of continuous thoughts does not reach the output. Hidden Error Awareness (2026) find that on explicit-CoT models probes read errors well but patching does not repair. LTO (2026) train a correctness classifier on latent states as a reward. We report a repair that aims a specific value through a specific slot using natural donor states, and the honest comparison against reading the explicit chain.

3 Setup

Models. Table 1 lists the checkpoints. All CODI-family models produce six thoughts after a begin-of-thought token; each thought's output state goes through a projector into the next input, and each thought also writes keys and values (K/V) into the cache that every later position attends to. Coconut has no projector: the thought *is* the last hidden state, fed back as the next input embedding.

Table 1: Checkpoints. Accuracies are our greedy GSM8K-test reproductions (1,319 problems).

Name	Base	Source	Paper	Ours
CODI GPT-2	GPT-2 (124M)	`zen-E/CODI-gpt2`	43.7	43.6
CODI-1B	LLaMA-3.2-1B-Instruct	`zen-E/CODI-llama3.2-1b-Instruct`	55.6	55.5
SIM-CoT	LLaMA-3.2-1B-Instruct	`internlm/SIM_COT-LLaMA3-CODI-1B`	56.1	55.7
Coconut	GPT-2 (124M)	`UdeSA/coconut-0.2B-gsm8k-aug` (retrain)	34.6	34.6

Datasets ("pools"). Every experiment needs problems whose solution is an equation chain, written as `«16-3-4=9» «9*2=18»`, because the twins recompute the chain. We use four pools: the GSM8k-Aug training split used by CODI itself (2,048 rows, plus 14,336 extra rows for long chains), the original GSM8K test set with its calculator annotations as the chain (1,206 usable rows, 854 with three or more steps), the original GSM8K train set (6,998 rows), and ASDiv (Miao et al., 2020) with its formula split into steps (1,490 rows, 86 with three or more steps). SVAMP (Patel et al., 2021) serves as a one-step control (300 rows). Problems with at most two steps are "short"; three or more are "long".

Outcome. For the CODI family we read the model's first answer token under the forced cue "The answer is:", which the released decoder uses, and also run native greedy decoding for headline conditions. Every rate comes with a 95% paired bootstrap interval over problem pairs (2,000 resamples). For Coconut, which has no forced cue, every outcome is native decoding.

4 Method

We describe the instruments in the order the audit runs them. The language is plain on purpose; the code is public and every step has a unit test.

4.1 Two ways a thought can act

A thought at position s affects the answer by two routes. Its *state* is the vector it hands to position $s+1$ through the projector. Its *K/V* are the keys and values it writes at its own cache position, which every later position reads through attention. We can edit either route alone: replace the state before it is projected (the "state" route), or overwrite the keys and values at that position after they are written (the "K/V" route), on any subset of layers. Replacing all six states *and* all six positions' K/V is identical to replacing the K/V alone, because the last state is discarded and everything a state does, it does through the K/V its successors write; we verified this identity exactly. Feeding a thought its own recorded state, or its own K/V, reproduces the original logits to the last bit, which is the test that the plumbing is right.

4.2 Twins: does a thought carry a value, or only disturb?

Swapping a thought from an unrelated problem breaks answers, but breaking is not carrying. To test carrying we build *twins*: for a problem, change one integer in the question and recompute the chain, so the twin's answer is the correct answer to the changed question. For example, "12 cucumbers" becomes "13 cucumbers" and the chain `«12*5=60» «54+60=114»` becomes `«13*5=65» «54+65=119»`. A number is eligible only if it appears exactly once in the question and once in the chain, is not itself a result, and the recomputed results stay non-negative integers. About half of GSM8k-Aug rows yield a twin.

Competence gate. We keep a pair only if the model solves both twins natively and at the first token, and their gold first tokens differ. Otherwise "the answer followed the swapped value" has no meaning.

Intervention and outcome. The two twins sit next to each other in the same batch, so their position ids match, and each is the other's donor. We replace one site in the recipient (one thought's state, or one position's K/V on a layer group, or whole tails) with the twin's and read the first answer token. The outcome is *target* if it becomes the twin's answer, *retain* if it stays the recipient's, *other* otherwise. The target rate is *transport*. Both directions are pooled.

Profiles. The *standard* profile draws 1,024 candidate twins from the pool in a fixed shuffled order and perturbs the first eligible number; most pairs are short chains. The *long* profile keeps only chains of three or more steps, perturbs the number that enters the chain latest, and draws from the extended pool. Both cap at 512 pairs. On the GSM8K test and train pools the caps are reached or the number of pairs is reported.

Locating the value. The model's own output head, applied to a thought's state, decodes a few numeric tokens. If the recipient's state at some thought decodes its own changed value and the twin's decodes the twin's, we say the value is *located* there. "Uniquely located" means at one thought only.

4.3 Masking: does the model need the thought?

Sufficiency is not necessity. For each thought we run two removals. *Mean*: replace its state by the average state over the pool, so nothing problem-specific reaches the next thought. *Mask*: hide its K/V from every later position through the attention mask, which needs a decoding path that carries the attention mask and explicit position ids through every step (the released LLaMA path drops the mask and we fixed that). We measure the accuracy drop in points, split by chain length, with intervals from a bootstrap over rows. Three preregistered checks: short chains survive any single removal within 5 points (K1), the carriers cost at least 10 points on long chains (K2), the last thought costs at most 2 points (K3).

4.4 What a probe is, and how it reads a thought

A thought is a vector of 768 numbers (GPT-2) or 2,048 numbers (the 1B models). A *probe* is a small model trained, after the fact, to predict a label from such a vector. It never touches the reasoning model; it only reads vectors we recorded. We use three readers.

1. **The model's own output head (no training).** The reasoning model ends with a matrix that turns a vector into scores over its vocabulary. Applying it to a thought's state asks "if this vector were about to be spoken, which token would it say?" We keep the numeric tokens and read the top few. This reader costs nothing and uses the model's own notion of a number; it is what the released decoder would do if it decoded the thought.
2. **A ridge probe (linear).** We fit a linear map W so that xW is close to the embedding of the target number's token, with a small penalty on W to avoid overfitting (ridge regression, closed form). To read, we map the state and pick the numeric token whose embedding is nearest. The target during training is the chain's penultimate intermediate result, the value the carriers hold.

3. **An MLP probe (non-linear).** A two-layer network (hidden width 256) trained to classify the state into one of about 900 numeric tokens, with weight decay, 300 full-batch steps and a fixed seed. It can read structure a linear map cannot.

Held-out reading. Probes are trained on four fifths of the recorded pairs and read the remaining fifth, five times over, with folds assigned by twin pair so a probe is never read on a problem whose twin it trained on. *Readability* is the held-out accuracy. A readability of 0.79 at thought 3 of CODI-1B means that, on four problems in five it had not seen, the probe named the right intermediate value from that one vector.

Why readability is not enough. A probe can read a value from a vector the model never uses, the way a bystander can read a note the recipient threw away. So we ask a second question with the twins. After a swap at thought s, the recipient holds the twin's state there. A monitor placed at s would read the twin's value and raise an alarm, "the model is working with value v' now". *Faithfulness* is the share of those alarms the behaviour confirms: how often the answer actually became the twin's. At CODI-1B thought 3 the probe reads at 0.79 and the alarm is confirmed 0.36 of the time; at thought 4 it reads at 0.84 and the alarm is confirmed 0.05 of the time. Both are readable; only one is a monitor.

Prompt-leakage probes. The same machinery, with a different label, tests what else a probe can read. We train probes to predict two things the question already fixes before any reasoning: the first operator in the chain and whether the problem has three or more steps. If these are readable from every thought, a probe's accuracy on a prompt-fixed property says nothing about the plan.

4.5 Monitors: readable, and faithful?

A monitor is a probe trained to read a value from a thought's state (Section 4.4). We train two families on the recorded twin states: a ridge map from the state to the embedding of the target number token, decoded by the nearest numeric token; and a two-layer MLP classifier over the numeric vocabulary. The target is the chain's penultimate intermediate result. Folds are assigned by pair so a twin never trains the probe that reads it. *Readability* is held-out accuracy. *Faithfulness* asks: when a monitor placed at this thought reads the twin's value from the twin's state after a swap (that is, raises an alarm), how often did the answer actually follow? A thought can be readable and never faithful.

4.6 The card

The driver runs the twins (both profiles), the masking audit, the two probe families and the bootstraps, and writes one card. Fixed thresholds: a *carrier* is a thought whose long-chain state transport is at least 0.15; a *store* is a position whose K/V transport is at least 0.15 while its state does not carry; the short-chain ceiling is 0.10; *necessary* means at least 10 points on long chains; a thought is *readable* at probe accuracy 0.30 and *readable-but-unfaithful* if its alarm is confirmed at most 5% of the time. Five checks: C1, no single site transports on short chains (position 0's K/V excepted, since it holds the question); C2, a carrier exists on long chains; C3, the last thought is inert, unfaithful and unnecessary; C4, readability is dissociated from faithfulness; C5, every carrier is necessary. The *regime* is *critical path* if C5 holds, *backup route* if carriers exist but C5 fails, *no transport* if there is no carrier. A long profile with fewer than 50 gated pairs does not decide carriers. The card records the GPU, library versions, dataset hash, and a SHA-256 fingerprint of every twin run's predictions, and a comparison tool flags disagreements across pools and sessions.

4.7 Prompt-leakage probes

To test whether a probe reads the plan or the prompt, we train probes on every thought to predict two properties the question already fixes: the first operator of the chain (four classes) and whether the chain has three or more steps. We report balanced accuracy against chance (0.25 and 0.50) and compare the best slot with the carriers.

4.8 Error detection from thoughts

For every GSM8K test problem, right or wrong, we record the thoughts and the native answer, apply the model's own output head to each thought (top-1 numeric value, its probability, the entropy, and

agreement with the answer and the other thoughts), and train logistic classifiers (five folds) to predict "the answer is wrong": carrier slots only, inert slots only, all slots, the answer's own first-token confidence, and a classifier on all raw state features. We report AUROC with bootstrap intervals.

4.9 Repair by transplanting a natural thought

The audit says the carrier thought holds the intermediate value and passes it on. Repair asks whether that channel can be used in reverse: if the model got a problem wrong, can we put the right intermediate value into the carrier and get the right answer out? We cannot write a number into a vector directly, since we do not know the code. But we can find a problem the model solved where the carrier already holds that number, and borrow its vector.

Worked example. The test problem asks for $(18 \times 3) + (12 \times 5)$; the chain is `«18*3=54»` `«12*5=60»` `«54+60=114»`, so the penultimate intermediate is $v^* = 60$. CODI-1B answers 108 and its carrier reads 48. In the bank of 2,048 training problems, the model's output head reads 60 at the carrier of 41 problems. We take the one whose carrier vector is most similar to the test problem's (cosine), copy that vector in at thought 3 while decoding the test problem, and read the new answer. If it is 114, the problem is *repaired*; if it is still 108, *unchanged*; if something else, *changed but wrong*.

Steps. (1) Decode the 2,048 bank problems once, recording all six thoughts and their K/V; read each carrier with the output head to index the bank by value. (2) Decode every test problem with three or more steps; keep the wrongly answered ones (up to 512) and 128 correctly answered ones as controls. (3) For each target, find bank entries whose carrier reads v^*; pick the nearest. (4) Decode the target again under each condition and classify the answer. (5) Report rates with bootstrap intervals, on the *primary stratum* (wrong targets whose own carrier reads something other than v^* and that have a donor) and on all wrong targets with a donor.

Controls, and what each rules out. A *random same-value* donor rules out the nearest-neighbour choice mattering. The *centroid* of all same-value bank vectors is a synthetic vector in the style of Cywiński et al. (2025); it tests whether a natural single vector is needed. A *random different-value* donor is the key control: if it repaired as often, the repair would be a disturbance that happens to shake a wrong answer loose, not the value. The *store K/V* and *state plus K/V* conditions test the second route. The donor's *whole K/V tail* is a ceiling on transport, not a repair. The 128 correct controls measure *harm*: how often the same transplant, with v^* set to the control's own penultimate value, breaks a right answer.

We record the thoughts of a bank of 2,048 training-pool problems and index each by the value the model's own output head reads at the carrier. For every wrongly answered test problem with three or more steps, we take the needed intermediate value v^* (the chain's penultimate result), find bank problems whose carrier reads v^*, and transplant the nearest one's carrier state; we decode again and ask whether the answer became correct. Controls: a random same-value donor, the centroid of same-value states (a synthetic vector, as in Cywiński et al. (2025)), a random different-value donor, the store position's K/V alone and with the state, the donor's whole K/V tail, and 128 correctly answered problems to measure harm. The *primary stratum* is wrong problems whose own carrier reads a value other than v^* and that have a donor.

Tool-supplied value. Every CODI checkpoint was also trained on explicit chains, so decoding the bare question makes it write its own equations. A calculator re-evaluates each one, propagating corrected results into later equations that reference them; the corrected last result is the tool's v^*. We report, as the honest ceiling, how often the explicit chain's own answer is right on the same failures.

4.10 Coconut adapter

We reimplemented Coconut's forward pass with the same hooks: six latent passes where each input embedding is the previous last hidden state, a state hook that may replace a thought before it is fed on, a K/V hook at each position, K/V masks, and batched cached decoding with left padding. Feeding a thought its own state or K/V reproduces every output exactly, and batch size does not change outputs. All Coconut outcomes are native answers after the `###` marker.

4.11 Preregistration and reproducibility

Every experiment was written into a ledger with its predictions before the run, and scored afterwards; eleven predictions were partly or wholly wrong and are reported as such. Runs executed on Kaggle T4 GPUs through the API from two accounts. Five reruns reproduced their twin predictions byte for byte. Code, cards, per-target outcomes and the ledger are in the public repository.

5 Results: the CODI family

5.1 Influence is not transport, and transport has a chain-length gate

Table 2 gives the central numbers. Swapping the whole K/V tail changes 60–75% of answers on all chains, yet on short chains no single thought moves more than 10% of answers to the twin's value. On long chains, one thought (GPT-2 and SIM-CoT: thoughts 1 and 3; CODI-1B: thought 3 alone) transports the value 0.20–0.36 of the time, and the K/V of the following position transport it at the same rate. This holds on the augmented training rows, the original test set and the original train set, with overlapping intervals.

Table 2: Transport (target rate) under a single-site swap, long chains. Columns are thought states $s_0..s_5$ and K/V positions $k_0..k_5$; the carrier/store pairs are bold. Short-chain transport never exceeds 0.10 at any single site. Pairs: 512 unless stated.

Model	Pool	s_0	s_1	s_2	s_3	s_4	s_5	k_0	k_1	k_2	k_3	k_4	k_5	kv_all
GPT-2	aug-train	.00	**.15**	.00	**.33**	.00	.00	.12	.00	**.16**	.00	**.30**	.00	.69
GPT-2	test (103 pairs)	.00	.13	.00	**.36**	.00	.00	.21	.00	.13	.00	**.33**	.00	.68
GPT-2	train-orig	.00	.14	.00	**.30**	.00	.00	.15	.00	.13	.00	**.29**	.00	.61
CODI-1B	aug-train	.04	.01	.01	**.34**	.04	.00	.04	.04	.01	.00	**.26**	.05	.67
CODI-1B	test (152 pairs)	.06	.01	.00	**.30**	.03	.00	.07	.06	.01	.00	**.24**	.03	.67
CODI-1B	train-orig	.06	.02	.00	**.35**	.03	.00	.07	.06	.02	.00	**.29**	.04	.65
SIM-CoT	aug-train	.04	**.19**	.04	**.25**	.04	.00	.13	.04	**.18**	.04	**.23**	.04	.73
SIM-CoT	test (168 pairs)	.03	**.22**	.05	**.20**	.04	.00	.22	.03	**.21**	.05	**.20**	.04	.72
SIM-CoT	train-orig	.03	**.21**	.05	**.23**	.05	.00	.17	.03	**.22**	.05	**.22**	.05	.66

The pattern is a ladder: an odd thought computes and projects into the next even position, which stores the result in its K/V, where the answer decode reads it. The pairing is trained, not fixed: (1→2, 3→4) in GPT-2 and SIM-CoT, (3→4) alone in CODI-1B. The sixth thought's state is discarded by the architecture and its K/V transport is at most 0.05 everywhere. On the one-step control SVAMP, no site transports more than 0.045 and the card reports that nothing is monitorable. On ASDiv the short-chain ceiling holds on all three models; its multi-step subset gave 17–30 pairs, too few to decide carriers, though the point estimates (0.12–0.15 at thought 3) point the same way.

5.2 Used is not needed

Table 3 gives the masking costs. No single thought costs more than 1.3 points on short chains in any model (K1 holds everywhere). On long chains the carriers cost GPT-2 6–20 points and SIM-CoT 4–12, but CODI-1B only 2–4 points, with upper bounds below 10 on every pool. CODI-1B uses thought 3 (transport 0.30–0.35) and does not need it: the *backup route* regime. The single most necessary site in GPT-2 and SIM-CoT is not a thought but position 0's K/V, which holds the question: hiding it costs 19–38 points (GPT-2) and 33–59 points (SIM-CoT) on long chains, and for SIM-CoT it alone transports 0.44–0.66 of short-chain twins. The last thought costs at most 1.9 points anywhere (K3 holds). GPT-2's carrier cost straddles the 10-point line across pools (6.5 [2.4, 10.6] on the test set, 11.7 [7.7, 16.0] and 12.5 [7.2, 17.9] on the others); we do not read a regime change into overlapping intervals.

5.3 Readable is not faithful

Table 4 compares what a probe reads with what the behaviour confirms. In every model the carriers are the only thoughts whose alarms are confirmed (0.17–0.40). Thoughts 4 and 5 are read at 0.4–0.8 by the ridge probe and confirmed at 0.00–0.05. The original check C4 ("the most readable slot is inert and readability anti-correlates with faithfulness") held for GPT-2 under the ridge probe but not under the MLP, and not for the 1B models: it is a probe-dependent statement and we withdraw it.

Table 3: Accuracy drop in points when one thought is removed (larger of mean-ablation and K/V mask), long chains; 95% bootstrap intervals over rows where available. "k_0" is the question register.

Model	Pool	k_0	1	2	3	4	5	all	Regime
GPT-2	aug-train	37.6	20.2 [14.4, 26.2]	12.2	12.5 [7.2, 17.9]	13.7	1.9	52.9	critical path
GPT-2	test	18.7	12.2 [7.9, 16.8]	9.2	6.5 [2.4, 10.6]	9.8	0.8	27.1	backup route
GPT-2	train-orig	29.5	18.3 [13.8, 23.2]	9.5	11.7 [7.7, 16.0]	12.9	1.1	37.0	critical path
CODI-1B	aug-train	3.4	2.7	1.5	2.7	4.6	0.8	38.8	backup route
CODI-1B	test	4.3	4.1	1.9	3.8 [1.3, 6.5]	4.3	1.1	28.5	backup route
CODI-1B	train-orig	3.2	7.2	3.2	2.3 [0.3, 4.3]	4.6	0.9	52.4	backup route
SIM-CoT	aug-train	58.6	10.6	11.4	9.1	8.4	3.0	76.0	backup route
SIM-CoT	test	32.5	9.8 [6.5, 13.3]	7.0	4.3 [2.2, 6.8]	5.7	1.9	44.2	backup route
SIM-CoT	train-orig	49.0	12.3 [8.3, 16.3]	11.7	9.2 [5.7, 13.2]	11.7	1.4	68.2	backup route

What holds on every card under both families is C4b: a thought exists that a probe reads at ≥ 0.30 whose reading the behaviour confirms $\leq 5\%$ of the time, and the last thought is such a slot in every model. Carrier faithfulness moves by at most 0.04 between probe families: faithfulness belongs to the slot, not to the probe.

Table 4: Monitor readability (ridge / MLP) and faithfulness (ridge / MLP) per thought, long chains, penultimate intermediate as the target. Carriers in bold.

Model	Pool	0	1	2	3	4	5
Readability ridge / MLP							
GPT-2	aug-train	.09/.10	**.33/.49**	.46/.48	**.40/.58**	.54/.56	.42/.57
GPT-2	test	.05/.03	.32/.36	.41/.23	**.40/.42**	.51/.27	.36/.40
CODI-1B	aug-train	.27/.17	.15/.06	.36/.24	**.79/.75**	.84/.79	.81/.73
CODI-1B	test	.19/.15	.06/.06	.16/.08	**.59/.52**	.69/.62	.68/.48
SIM-CoT	aug-train	.25/.29	**.62/.61**	.69/.56	**.72/.67**	.74/.62	.57/.44
SIM-CoT	test	.10/.12	**.44/.38**	.50/.28	**.56/.44**	.54/.32	.32/.19
Faithfulness ridge / MLP (alarm confirmed by behaviour)							
GPT-2	aug-train	.00/.00	**.19/.17**	.00/.00	**.40/.36**	.00/.00	.00/.00
GPT-2	test	.00/.00	.09/.08	.00/.00	**.29/.31**	.00/.00	.00/.00
CODI-1B	aug-train	.02/.01	.01/.00	.02/.02	**.36/.36**	.05/.05	.00/.00
CODI-1B	test	.04/.05	.06/.06	.00/.00	**.34/.35**	.02/.02	.00/.00
SIM-CoT	aug-train	.02/.02	**.21/.20**	.02/.01	**.28/.29**	.03/.03	.00/.00
SIM-CoT	test	.03/.03	**.26/.30**	.03/.01	**.26/.28**	.02/.04	.00/.00

6 Results: three uses of the map

6.1 A monitor can read the prompt instead of the plan

Table 5 shows that a probe on *any* thought, position 0 and the unused last thought included, decodes whether the problem is multi-step at 0.76–0.95 balanced accuracy and the chain's first operator at 0.50–0.87 against 0.25. The best slot for the operator is never a carrier: it is the question register or a store position. Only the computed value separates the slots. A monitor evaluated on a property the prompt already fixes (a trigger word, a task type, a difficulty) can therefore score well while reading nothing the model computed. For a trigger-in-the-prompt backdoor evaluation this is a confound, and the fix is cheap: run the same probe on a prompt-only control at the same slot.

Table 5: Balanced accuracy of a probe decoding two prompt-fixed properties from each thought (ridge / MLP), standard profile. Chance 0.25 (operator) and 0.50 (long chain).

Model	Property	0	1	2	3	4	5	best / carriers
GPT-2	operator	.63/.76	.55/.66	.63/.74	.60/.67	.62/.76	.60/.68	4 / {1,3}
GPT-2	long chain	.81/.91	.81/.86	.84/.92	.79/.84	.85/.91	.80/.84	2 / {1,3}
CODI-1B	operator	.85/.86	.85/.87	.81/.84	.72/.72	.74/.73	.74/.75	1 / {3}
CODI-1B	long chain	.89/.91	.92/.93	.91/.94	.91/.91	.90/.92	.92/.92	2 / {3}
SIM-CoT	operator	.84/.80	.71/.71	.73/.74	.74/.73	.74/.75	.72/.72	0 / {1,3}
SIM-CoT	long chain	.93/.95	.93/.94	.93/.95	.93/.93	.92/.94	.91/.93	0 / {1,3}

6.2 Thoughts do not catch errors better than confidence

Table 6 is a negative result for the application and a positive one for the map. Nothing read from the thoughts predicts a wrong answer better than the answer's own first-token confidence (0.89–0.91 AUROC), not even 12,288 raw state features, and adding the carrier readout to confidence gains under 0.01. But on both 1B models the error signal inside the thoughts sits at the carrier the swaps found and nowhere else (carrier 0.82 vs inert 0.60 on CODI-1B), an independent confirmation of the placement from a task it was never fitted to. On GPT-2 the uncertainty is in every slot.

Table 6: AUROC for "the native answer is wrong" on the full GSM8K test set (1,319 rows).

Model	Wrong	Confidence	Carriers	Inert	All slots	Raw states
GPT-2	746	0.889	0.850	0.853	0.851	0.859
CODI-1B	587	0.912	0.821	0.596	0.827	0.835
SIM-CoT	584	0.913	0.754	0.650	0.799	0.836

6.3 The carrier can be written to

Table 7 shows repair rates. With the needed value from the gold chain, transplanting the nearest same-value natural thought into the carrier turns 15–23% of failures into the gold answer, against 2.6–3.6% for a different-value donor; the intervals do not overlap. At the 1B models the same transplant harms only 5–7% of correct answers; at GPT-2, 19%. Two preregistered predictions were wrong in useful ways: the centroid of same-value states repairs as well as a retrieved natural one, so retrieval buys nothing over averaging; and adding the store position's K/V *lowers* repair, because the donor's K/V carry the donor's problem as well as the value. With the value from the model's own explicit chain plus a calculator, repair survives at CODI-1B (18.8%, 83% of the gold rate) and weakens at the other two (9.2%, 8.4%). On every model the explicit chain read directly is right on more of the same failures (18.6–22.7%) than injection recovers, so this is a mechanism result: the audited thought is a writable channel, confirmed by two independent sources of the value, not a method that beats reading the chain.

Table 7: Repair rate (%) on wrongly answered $\geq$ 3-step GSM8K test problems whose carrier read the wrong value and that have a donor; harm (%) on correctly answered controls. 95% bootstrap intervals for the main column.

Value from	Model	n	Nearest donor	Random same	Centroid	Different value	Store K/V	State+K/V	Harm
gold chain	GPT-2	357	15.1 [11.5, 19.0]	17.6	20.2	3.6	11.5	12.3	18.7
gold chain	CODI-1B	266	22.6 [17.7, 27.8]	21.8	23.7	2.6	10.2	13.2	7.0
gold chain	SIM-CoT	273	15.4 [11.4, 19.8]	9.9	13.2	2.6	4.0	4.4	5.2
calculator	GPT-2	282	9.2 [6.0, 12.8]	9.2	12.4	3.5	7.1	7.1	16.8
calculator	CODI-1B	208	18.8 [13.9, 24.0]	14.4	16.8	3.8	10.1	10.6	6.0
calculator	SIM-CoT	214	8.4 [5.1, 12.1]	7.0	8.9	3.3	2.3	3.3	10.4

7 Results: Coconut is the opposite case

Table 8 runs the same audit on Coconut. The predicted pairing from its curriculum (two thoughts per step) appears exactly: thought 0's state and position 1's K/V transport at the same rate, as do 2 and 3, and 4 and 5. Everything else inverts. Coconut transports on short chains (0.45 at the first thought), so the chain-length gate is a CODI property. Its long-chain transport is 0.52, higher than any CODI model. Its last position is in use: the state is discarded, but its K/V transport 0.15 and hiding them costs 14 points. Every pair is necessary: removing a single thought costs 45–49 points. And it cannot be written to: a same-value donor repairs 5.6% against 3.1% for a different value, while harming 29% of correct answers, because a Coconut thought transplanted from another problem carries that problem. Same six slots, same task, opposite monitoring profile.

Table 8: Coconut (GPT-2 retrain, gate 34.57%). Transport on short (standard, $n = 748$) and long ($n = 1{,}024$) chains; masking cost on long chains; repair on 360 primary targets.

	s_0	s_1	s_2	s_3	s_4	s_5	k_1	k_3	k_5
transport, short	**.45**	.07	.03	.01	.02	.00	**.45**	.03	.02
transport, long	.19	.12	**.52**	.10	.14	.00	.19	**.52**	.15
masking cost, long (pts)	49 (mean)	48 (mask)	45 (mean)	46 (mask)	16 (mean)	14 (mask)			

repair: nearest same-value donor 5.6%, different value 3.1%, centroid 9.4%; harm 29% (donor state), 88% (whole tail)

8 Discussion

What this means for monitoring latent reasoning. For the CODI family a monitor can be placed where it means something, at the carrier, and the audit finds that slot in about an hour. Placing it at the most readable thought is a mistake, since the most readable thoughts are routinely the ones the behaviour never confirms, and any thought will happily report the prompt. For Coconut-style models the picture is reversed: every even thought is both used and needed, and readability tracks use. The practical rule is to run the audit before trusting a probe, and to include a prompt-only control in every monitor evaluation.

Can the repair raise accuracy? Not as a method, and we say so plainly. We costed three policies on the 838 long-chain test problems using the per-target outcomes of the calculator runs (Table 9). Applying the repair to every problem gains 2.8 points on CODI-1B and loses points on the other two, because the harm to correct answers outweighs the repairs. Applying it only when the model's explicit chain disagrees with its latent answer (a trigger that needs no answer key) gains 3.4 points on CODI-1B, 1.1 on SIM-CoT and nothing on GPT-2. But the obvious alternative, switching to the explicit chain's answer on the same disagreement, gains 5.3, 3.2 and 5.6 points, and it needs no transplant. Repair also costs an explicit decode to obtain the value, so it is not cheaper either. What the repair shows is a property of the model, a writable channel at a known slot, not a better way to get answers out of it.

Table 9: Net accuracy change (points) on the 838 long-chain GSM8K test problems under three policies, scaled from the repair runs' outcomes (calculator-supplied value). Latent accuracy: GPT-2 35.6, CODI-1B 49.6, SIM-CoT 50.1.

Policy	GPT-2	CODI-1B	SIM-CoT
Transplant the donor thought on every problem	-1.8	$+2.8$	-2.0
Transplant only when the explicit chain disagrees with the latent answer	-0.1	$+3.4$	$+1.1$
Switch to the explicit chain's answer when it disagrees	$+5.6$	$+5.3$	$+3.2$

Why the two families differ. CODI distils a teacher into thoughts through a projector and drops the chain's last step during training; what emerges is a narrow channel that holds one intermediate at one slot, with the question kept in position 0's memory and a backup route around the thoughts in the larger model. Coconut replaces steps with thoughts by a curriculum, step by step; what emerges is a thought sequence that is the computation, dense and necessary. Step supervision (SIM-CoT) moves CODI part of the way toward the Coconut profile: more necessary thoughts and a heavier reliance on the question register.

Limitations. All results are on grade-school arithmetic; ASDiv transfers only the short-chain and terminal findings because its multi-step subset is small, and we found no further dataset with equation chains and small numbers. The Coconut checkpoint is a third-party retrain at GPT-2 scale. The repair's needed value came from the gold chain or from the model's own chain; a deployable repairer would need a tool that supplies it, and reading that chain directly is already better. Outcomes are first answer tokens for the CODI family, with native decoding for headline conditions. Two probe families were tried, not more.

Errata we made along the way. We first gated the CODI-1B reproduction against 51.9%, which is an ablation row in its paper; the main figure is 55.6% and the checkpoint matches it. We first wrote

that C4 held on all three training-pool cards; it held on one. We first read GPT-2's cost difference between pools as a regime change; the intervals overlap. All three are recorded in the ledger with the correction.

9 Conclusion

Hidden thoughts in CODI-style models are optional, narrow channels: they carry a value only on hard problems, through one trained slot, they mislead readers, and they can be edited from outside. Coconut-style thoughts are necessary, entangled and uneditable. A cheap, inference-only audit tells you which kind you have, where a monitor would mean something, and what a monitor's accuracy could be measuring instead.

Reproducibility statement

The repository `github.com/0x0shephard/latent-reasoning` contains the audit driver, every runner, 80 unit tests for the audit code, the pinned Kaggle notebooks, 14 cards, the per-target outcomes of every repair and error-detection run, and the ledger (`docs/RESEARCH_CONTEXT_LEDGER.md`, sections 108–127) with each preregistration and its scoring. Checkpoints are pinned by revision and SHA-256. Each card records the GPU, library versions and dataset hash. Five reruns, on another account and day, reproduced the twin predictions byte for byte.

References

D. Aswal, T. Palmeira Ferraz, Y. Zhou, and M. Peyrard. Observable patterns are not explanations: A causal-geometric analysis of latent reasoning models. *arXiv:2606.12689*, 2026.

S. Chang et al. Unlocking the black box of latent reasoning: An interpretability-guided approach to intervention. In *ACL*, 2026. arXiv:2606.01243.

B. Cywiński, B. Bussmann, A. Conmy, J. Engels, N. Nanda, and S. Rajamanoharan. Can we interpret latent reasoning using current mechanistic interpretability tools? *Alignment Forum*, December 2025.

Ding, Huang, and Yang. SCIT. In *EMNLP*, 2026. arXiv:2608.27265.

Ferreira et al. Does out-of-sight equal out-of-mind in CoT monitorability? *arXiv:2608.04928*, 2026.

S. Hao et al. Training large language models to reason in a continuous latent space. *arXiv:2412.06769*, 2024.

Hidden error awareness in chain-of-thought reasoning: The signal is diagnostic, not causal. *arXiv:2605.09502*, 2026.

G. Huang and L. Qi. When steering fails in latent reasoning: A latent-to-language transition gap. *arXiv:2609.21662*, 2026.

Jin, Yang, and Wang. Final checkpoints are not enough: Analyzing latent reasoning faithfulness along training trajectories. *arXiv:2607.06648*, 2026.

Li et al. Dynamics within latent chain-of-thought: An empirical study of causal structure. In *ICML*, 2026. arXiv:2602.08783.

Latent thinking optimization: Your latent reasoning language model secretly encodes reward signals in its latent thoughts. In *ICLR*, 2026. arXiv:2509.26314.

S.-Y. Miao, C.-C. Liang, and K.-Y. Su. A diverse corpus for evaluating and developing English math word problem solvers. In *ACL*, 2020.

Open problems in mechanistic interpretability. *arXiv:2501.16496*, 2025.

Decodability is not causality: Dissociating probe readouts from behavioral drivers via SAE decomposition. *arXiv:2609.18080*, 2026.

Single-position intervention fails: Distributed output templates drive in-context learning. *arXiv:2605.04061*, 2026.

A. Patel, S. Bhattamishra, and N. Goyal. Are NLP models really able to solve simple math word problems? In *NAACL*, 2021.

Ramjee et al. Ulterior motives: Detecting misaligned reasoning in continuous thought models. In *ICLR*, 2026. arXiv:2604.23460.

Z. Shen et al. CODI: Compressing chain-of-thought into continuous space via self-distillation. In *EMNLP*, 2025. arXiv:2502.21074.

X. Wei et al. SIM-CoT: Supervised implicit chain-of-thought. In *ICLR*, 2026. arXiv:2509.20317.